causal_lm.py — technical

`src/moe_pruning/tasks/causal_lm.py · 109 lines · moe_pruning.tasks.causal_lm`

Purpose

Decoder-only objective.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class CausalExample`

lines 28-31 · dataclass

One evaluation document as token ids (labels = input ids).

Field	Type	Default
`example_id`	int	
`input_ids`	list[int]	
`labels`	list[int]	

`class CausalLMTask`

lines 34-109 · bases TaskObjective

Decoder-only objective with next-token evaluation.

`CausalLMTask.tokenize_calibration`

```
def tokenize_calibration(self, records, tokenizer, max_source_length, max_target_length)
```

lines 47-54

`human` is the input/prompt, `response` the teacher-forced target.

1. Prompt ids: chat template with one user turn and generation prompt if the tokenizer has a template, else `User: {human}\nAssistant: ` ; truncated to `MAX_SOURCE_LENGTH`.
2. Response ids without special tokens, truncated to `MAX_TARGET_LENGTH - 1`, plus EOS.
3. input_ids = prompt + response; labels = -100 for the prompt, response ids for the rest.

`CausalLMTask.collate`

```
def collate(self, items, pad_token_id)
```

lines 56-57

Right-padded batch with per-stack real-token masks.

`CausalLMTask.eval_settings`

```
def eval_settings(self, cfg) -> dict
```

lines 59-60 · settings MAX_SOURCE_LENGTH

Task-specific settings that determine the evaluation examples.

`CausalLMTask.build_eval_example`

```
def build_eval_example(self, example_id, text, tokenizer, cfg)
```

lines 62-64 · settings MAX_SOURCE_LENGTH

One evaluation example (must expose `example_id`, `input_ids`, `labels`).

1. Tokenize the document, keep the first `MAX_SOURCE_LENGTH` tokens; labels = ids.

`CausalLMTask.token_nll`

```
def token_nll(self, logits, labels)
```

lines 70-77

Per-example (sum of token NLL, number of scored tokens).

1. Shift: logits `[:, :-1]` predict labels `[:, 1:]`; then masked cross-entropy as in the seq2seq task.

`CausalLMTask.predictions`

```
def predictions(self, logits, labels)
```

lines 79-84

Per-example argmax token at every scored position (same order as `token_nll`).

`CausalLMTask.qualitative`

```
def qualitative(self, model, tokenizer, examples, device, cfg)
```

lines 87-109

`[{"example_id", "inputs": {label: text}, "prediction"}]` from greedy generation.

1. Prompt = first min(64, len/2) tokens; greedy 32 new tokens; reference = the next 32 document tokens.

Module constants

Name	Value
`PROMPT_TEMPLATE`	'User: {human}\nAssistant: '
`QUALITATIVE_PROMPT_TOKENS`	64
`QUALITATIVE_NEW_TOKENS`	32